

Disciplina:

# **Análise de Imagem e Visão Computacional**

Professor: Octavio Santana



Aula 6

# **Deteccção de Objetos e Reconhecimento de Faces**

Professor: Octavio Santana



# Objetivos da Aula

**Ao final desta aula, os alunos serão capazes de:**

1. Conceitos básicos de detecção de objetos
2. Rotulagem de dados em problemas de detecção de objetos
3. YOLO - Modelo de Detecção de Objetos em tempo real
4. Uso, Treinamento e Avaliação do modelo YOLO com ultralytics.

# Conceitos básicos de detecção de objetos

# Bounding Box (Caixa delimitadora)

Uma bounding box é um retângulo que delimita a região onde um objeto está localizado em uma imagem.

## Tipos de Representação:

- Coordenadas Absolutas ( $x, y, w, h$ ):
  - $(x, y)$  = canto superior esquerdo da caixa.
  - $w$  = largura da caixa.
  - $h$  = altura da caixa.
- Coordenadas Normalizadas ( $x\_center, y\_center, w, h$ ):
  - $(x\_center, y\_center)$  = centro da bounding box (valores entre 0 e 1, relativos à imagem).
  - $w, h$  = largura e altura normalizadas.
- Formato COCO ( $x1, y1, x2, y2$ ):
  - $(x1, y1)$  = canto superior esquerdo.
  - $(x2, y2)$  = canto inferior direito.



# Intersection over Union (IoU)

O IoU mede a sobreposição entre duas bounding boxes (predição e ground truth). É usado para avaliar a precisão da detecção.

## Cálculo do IoU:

- Calcule a área de interseção (intersection) entre as duas caixas.
- Calcule a área de união (union = área\_box1 + área\_box2 - intersection).
- $\text{IoU} = \text{intersection} / \text{union}$

## Interpretação:

- $\text{IoU} = 1 \rightarrow$  Caixas idênticas.
- $\text{IoU} = 0 \rightarrow$  Nenhuma sobreposição.
- Threshold comum: 0.5 (considera-se uma detecção válida se  $\text{IoU} \geq 0.5$ ).



# Intersection over Union (IoU)

## Implementação no Python

```
def calculate_iou(box1, box2):  
    # box1 e box2 no formato [x1, y1, x2, y2]  
    x1 = max(box1[0], box2[0])  
    y1 = max(box1[1], box2[1])  
    x2 = min(box1[2], box2[2])  
    y2 = min(box1[3], box2[3])  
  
    intersection = max(0, x2 - x1) * max(0, y2 - y1)  
  
    area_box1 = (box1[2] - box1[0]) * (box1[3] - box1[1])  
    area_box2 = (box2[2] - box2[0]) * (box2[3] - box2[1])  
    union = area_box1 + area_box2 - intersection  
  
    return intersection / union if union != 0 else 0
```

# Confiança e pontuação de classe

## Confiança (confidence):

- Probabilidade de a bounding box conter um objeto.

## Pontuação de Classe (class score):

- Probabilidade de o objeto pertencer a uma classe específica (ex: "cachorro", "gato").

## Exemplo:

- Uma detecção pode ter:
  - Confiança = 0.9 (90% de chance de haver um objeto na caixa).
  - Class scores = [0.1, 0.8, 0.1] (80% de chance de ser a classe 2).



# Mean Average Precision (mAP)

O mAP (Mean Average Precision) é a média das Precisões Médias (AP) calculadas para cada classe de objeto em um conjunto de dados. Ele avalia:

- Precisão da classificação: Se o modelo está identificando corretamente a classe do objeto.
- Precisão da localização: Se a bounding box prevista está alinhada com a verdade terrestre (ground truth) .

## Por que o mAP é importante?

- Métrica única: Resume o desempenho do modelo em um único número, facilitando comparações.
- Balanceamento entre precisão e recall: Considera tanto falsos positivos (precisão) quanto falsos negativos (recall) .
- Aplicações críticas: Usada em veículos autônomos, diagnósticos médicos e vigilância, onde erros podem ter consequências graves .

# Mean Average Precision (mAP)

O cálculo do mAP envolve várias etapas.

Passo 1 - Calcular precisão e recall

- **Precisão (P)**: Proporção de detecções corretas entre todas as previsões positivas.  $P = TP / (TP + FP)$
- **Recall (R)**: Proporção de objetos reais detectados corretamente.  $R = TP / (TP + FN)$

Onde:

- **TP** (Verdadeiros Positivos): Detecções corretas ( $IoU \geq \text{limiar}$ ).
- **FP** (Falsos Positivos): Detecções incorretas ou redundantes.
- **FN** (Falsos Negativos): Objetos reais não detectados .

Passo 2 - Curva Precisão-Recall (PR Curve)

- Para cada classe, ordena-se as detecções por confiança e traça-se a precisão em função do recall.
- Quanto maior a área sob a curva (AUC), melhor o desempenho .

Passo 3 - Calcular AP (Average Precision)

- AP é a área sob a curva PR para uma única classe.
- Pode ser aproximada por interpolação ou cálculo numérico .

# Mean Average Precision (mAP)

Passo 4 - Calcular mAP

- mAP é a média das APs de todas as classes.

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i$$

onde (N) é o número de classes .

# Variantes do mAP

## mAP@0.5 (mAP50)

- Calculado com  $\text{IoU} \geq 0.5$  (uma detecção é considerada correta se sobrepõe em pelo menos 50% com a ground truth).
- Métrica padrão em benchmarks como PASCAL VOC.

## mAP@0.5:0.95 (mAP50-95)

- Média do mAP calculado em diferentes limiares de IoU (de 0.5 a 0.95, em incrementos de 0.05).
- Mais rigoroso, usado no COCO para avaliar modelos em cenários de alta precisão.

## Interpretação do mAP

- mAP alto ( $>0.5$ ): Modelo robusto, com boa detecção e localização.
- mAP baixo ( $<0.3$ ): Pode indicar problemas como:
  - Baixa precisão (muitos FPs): Ajustar limiar de confiança ou melhorar treino.
  - Baixo recall (muitos FNs): Aumentar dados de treino ou usar técnicas como image tiling para objetos pequenos.
- Diferenças entre classes: Se algumas classes têm AP muito baixo, pode ser necessário balancear o dataset ou usar aumento de dados.

# Non-Maximum Suppression (NMS)

Quando um modelo detecta múltiplas bounding boxes para o mesmo objeto, o NMS filtra as detecções redundantes, mantendo apenas a mais confiável.

## Funcionamento:

- Ordena as detecções por confiança.
- Seleciona a caixa com maior confiança e remove outras caixas com  $\text{IoU} > \text{threshold}$  (ex: 0.5).
- Repete o processo até que todas as caixas sejam processadas.

## Exemplo em python

```
def nms(boxes, scores, iou_threshold=0.5):  
    keep = []  
    idxs = np.argsort(scores)[::-1] # Ordena por confiança  
  
    while len(idxs) > 0:  
        i = idxs[0]  
        keep.append(i)  
  
        # Calcula IoU com outras caixas  
        ious = [calculate_iou(boxes[i], boxes[j]) for j in idxs[1:]]  
  
        # Mantém apenas caixas com IoU < threshold  
        idxs = idxs[1:][np.array(ious) < iou_threshold]  
  
    return keep
```



# **Rotulagem de dados em problemas de detecção de objetos**



# O que é rotulagem e por que é importante?

A rotulagem (annotation) é o processo de marcar objetos em imagens/vídeos com:

- Bounding boxes (retângulos delimitadores).
- Classes (ex: cachorro, carro).

Porque é fundamental?



- Modelos de detecção (YOLO, Faster R-CNN) dependem de dados rotulados para aprender.
- Anotações imprecisas levam a falsos positivos/negativos e baixo mAP.

# Ferramentas de Rotulagem

## LabelImg (Open Source)

Indicado para projetos pequenos ou iniciantes.

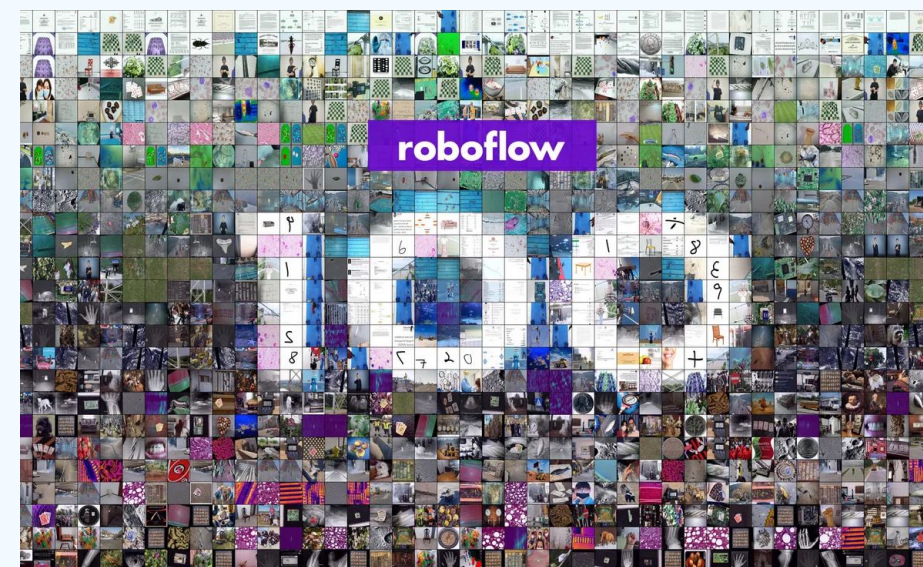
### Como usar?

- Instalação

```
pip install labelImg  
labelImg # Abre a interface gráfica
```

- Fluxo de trabalho:

- Abra uma imagem.
- Use W para criar uma bounding box.
- Atribua uma classe (ex: cachorro).
- Salve no formato YOLO (.txt) ou Pascal VOC (.xml).



### Vantagens:

- ✓ Simplicidade.
- ✓ Suporte a múltiplos formatos.

### Limitações:

- ✗ Sem colaboração em equipe.
- ✗ Sem versionamento de dados.



# Ferramentas de Rotulagem

## Roboflow (Plataforma Cloud)

Indicado para projetos grandes, times ou fluxos automatizados.

### Funcionalidades

- Upload de imagens (arrastar e soltar).
- Rotulagem assistida por IA (auto-annotation).
- Pré-processamento: Redimensionamento, aumento de dados (data augmentation).
- Exportação: Gera arquivos prontos para YOLO, TensorFlow, etc.

### Exemplo de uso

```
from roboflow import Roboflow

rf = Roboflow(api_key="SUA_API_KEY")
project = rf.workspace().project("meu-projeto")
dataset = project.version(1).download("yolov8")
```

### Vantagens:

- ✓ Colaboração em tempo real.
- ✓ Ferramentas de qualidade (validação de anotações).
- ✓ Geração automática de datasets balanceados.

### Limitações:

- ✗ Versão gratuita tem limites de armazenamento.

# Formatos de Anotação

## Formato YOLO

Cada imagem tem um arquivo .txt com linhas no formato:

```
<class_id> <x_center> <y_center> <width> <height> # Valores normalizados (0-1)
```

Exemplo

```
0 0.45 0.32 0.10 0.20 # Classe 0 (cachorro) no centro (45%, 32%)
```

## Formato COCO

Armazena anotações em um único JSON estruturado:

```
{  
  "images": [{"id": 1, "file_name": "imagem1.jpg"}],  
  "annotations": [{"id": 1, "image_id": 1, "bbox": [x, y, w, h]}]  
}
```

## Formato Pascal VOC

Usa arquivos XML por imagem

```
<object>  
  <name>cachorro</name>  
  <bndbox>  
    <xmin>100</xmin>  
    <ymin>200</ymin>  
    <xmax>300</xmax>  
    <ymax>400</ymax>  
  </bndbox>  
</object>
```

## Dicas para rotulagem de alta qualidade

- Consistência: Sempre use a mesma classe para objetos similares.
- Bounding boxes justas: Ajuste para cobrir todo o objeto, mas sem fundo desnecessário.
- Ocasões difíceis:
  - Objetos parcialmente visíveis → Rotule apenas a região visível, não tente extrapolar.
  - Pequenos objetos → Use zoom para maior precisão.
- Validação humana: Revise 10-20% das anotações aleatoriamente.

# **YOLO - Modelo de Detecção de Objetos em tempo real**



# YOLO (You Only Look Once)

O **YOLO** é uma das arquiteturas mais populares para detecção de objetos em tempo real, combinando alta velocidade e precisão. Diferente de modelos como **R-CNN** e **Faster R-CNN**, que usam pipelines complexos (proposta de regiões + classificação), o YOLO trata a detecção como um problema de regressão única, processando a imagem inteira em uma única passagem pela rede neural.

## Principais Características:

- Arquitetura otimizada para maior velocidade e precisão.
- Detecção sem âncoras (anchor-free) – prevê diretamente o centro do objeto em vez de ajustar caixas pré-definidas.
- Suporte a múltiplas tarefas (detecção, segmentação, classificação).
- Treino eficiente com aumento de dados (data augmentation) e otimização de loss functions.

# Como o YOLO Detecta e Classifica Objetos

## Passo 1: Divisão da Imagem em uma Grade (Grid)

O YOLO divide a imagem em uma grade  $S \times S$  (ex:  $13 \times 13$  ou  $26 \times 26$ ). Cada célula da grade é responsável por prever objetos cujo centro está dentro dela.

## Passo 2: Extração de Características (Backbone + Neck)

- Backbone (C2f Module):
  - Extrai características de baixo, médio e alto nível da imagem (como bordas, formas e texturas).
  - Substitui o CSPDarknet por blocos C2f (mais eficientes computacionalmente).
- Neck (FPN + PAN):
  - Combina características de diferentes resoluções para melhor detecção de objetos grandes e pequenos.

# Como o YOLO Detecta e Classifica Objetos

## Passo 3: Cabeça de Detecção (Head)

A cabeça do YOLO (head) gera três tipos de saídas para cada célula da grade:

- Coordenadas da Bounding Box ( $x, y, w, h$ )
  - $(x, y)$  = centro do objeto (relativo à célula).
  - $(w, h)$  = largura e altura (normalizadas em relação à imagem).
- Confiança da Detecção (objectness score)
  - Probabilidade de haver um objeto naquela região.
- Probabilidade de Classe (class scores)
  - Probabilidades de o objeto pertencer a cada classe (ex: "carro", "pessoa").

## Passo 4: Pós-Processamento (NMS - Non-Maximum Suppression)

- Elimina detecções redundantes (várias caixas para o mesmo objeto).
- Mantém apenas as previsões com maior confiança e IoU (Intersection over Union) alto.

## Fluxo completo de detecção

| Etapa             | Descrição                                                                  |
|-------------------|----------------------------------------------------------------------------|
| Pré-processamento | Redimensiona a imagem para o tamanho de entrada (ex: 640×640).             |
| Forward Pass      | Passa a imagem pela rede (backbone + neck + head) para obter as previsões. |
| Decodificação     | Converte as saídas da rede em coordenadas reais de bounding boxes.         |
| Filtragem (NMS)   | Remove detecções duplicadas e mantém apenas as mais confiáveis.            |

# Treinamento do YOLO

O YOLO é treinado usando:

- Função de Perda (Loss Function) que combina:
  - Perda de Localização (CloU Loss) – mede a precisão da bounding box.
  - Perda de Classificação (BCE Loss) – mede a acurácia da classe.
  - Perda de Confiança (Objectness Loss) – penaliza falsos positivos/negativos.
- Aumento de Dados (Data Augmentation) como Mosaic Augmentation (mistura 4 imagens para melhor generalização).



# **Uso, Treinamento e Avaliação do YOLO com Ultralytics**

# Instalação e Configuração

## Requisitos:

- Python  $\geq 3.8$
- GPU (recomendado) com CUDA (para aceleração)

## Instalação:

```
pip install ultralytics # Instala a biblioteca Ultralytics  
pip install torch torchvision # Instala PyTorch (opcional, mas recomendado para GPU)
```

## Verificação:

```
import ultralytics  
ultralytics.checks() # Verifica dependências e GPU
```

# Detecção de Objetos (Inferência)

## Modelos disponíveis ([link](#))

- YOLOv8: yolov8n.pt (nano), yolov8s.pt (small), yolov8m.pt (medium), etc.
- YOLO-World: yolov8s-world.pt (para detecção com prompts textuais).

## Exemplo Detecção em Imagem

```
from ultralytics import YOLO

# Carrega o modelo (ex: YOLOv8 nano)
model = YOLO("yolov8n.pt")

# Executa detecção
results = model.predict("imagem.jpg", save=True, conf=0.5)

# Mostra resultados
results[0].show()
```

## Opções do `predict()`

| Parâmetro           | Descrição                              | Exemplo                         |
|---------------------|----------------------------------------|---------------------------------|
| <code>source</code> | Caminho da imagem, vídeo ou webcam (0) | <code>source="video.mp4"</code> |
| <code>conf</code>   | Limiar de confiança (0-1)              | <code>conf=0.7</code>           |
| <code>save</code>   | Salva resultados                       | <code>save=True</code>          |
| <code>imgsz</code>  | Tamanho da imagem (pixels)             | <code>imgsz=640</code>          |

# Treinamento Personalizado

## Preparação do Dataset

- Formato recomendado: YOLO format (pastas train/images, train/labels, val/images, etc.).
- Cada imagem deve ter um arquivo .txt com anotações no formato:

```
<class_id> <x_center> <y_center> <width> <height> # Valores normalizados (0-1)
```

## Treinamento Básico

```
from ultralytics import YOLO

# Carrega um modelo pré-treinado (ex: yolov8n)
model = YOLO("yolov8n.pt")

# Treina o modelo
results = model.train(
    data="caminho/para/dataset.yaml", # Arquivo YAML com paths
    epochs=100,                       # Número de épocas
    batch=16,                         # Tamanho do lote
    imgsz=640,                       # Resolução de entrada
    name="meu_treinamento"           # Nome do experimento
)
```

## Arquivo dataset.yaml

```
train: caminho/para/train/images
val: caminho/para/val/images
names:
  0: cabo_de_rede
  1: roteador
```

## Monitoramento

- Os logs são salvos em runs/detect/meu\_treinamento/.
- Use TensorBoard:

```
tensorboard --logdir runs/detect
```



# Avaliação do Modelo

## Métricas Principais

- mAP50 (mAP@0.5): Precisão média em  $\text{IoU} \geq 0.5$ .
- mAP50-95: Média do mAP para IoU de 0.5 a 0.95.
- Precision (P): Proporção de detecções corretas.
- Recall (R): Proporção de objetos detectados.

## Código para Avaliação

```
from ultralytics import YOLO

model = YOLO("melhor_modelo.pt") # Carrega o modelo treinado
metrics = model.val()             # Avalia no conjunto de validação

print(f"mAP50: {metrics.box.map50}")
print(f"mAP50-95: {metrics.box.map}")
```

## Saída Esperada

| Class        | Images | Instances | P    | R    | mAP50 | mAP50-95 |
|--------------|--------|-----------|------|------|-------|----------|
| cabo_de_rede | 100    | 150       | 0.85 | 0.78 | 0.82  | 0.45     |
| roteador     | 100    | 120       | 0.90 | 0.85 | 0.88  | 0.50     |



## Conclusões

- Compreendemos os fundamentos da detecção de objetos, diferenciando-a de outras tarefas de visão computacional, como classificação e segmentação.
- Aprendemos a importância da rotulagem precisa, utilizando bounding boxes e identificadores de classe para treinar modelos de forma eficaz.
- Exploramos a arquitetura e funcionamento do YOLO, um dos modelos mais rápidos e eficientes para detecção em tempo real.
- Executamos o pipeline completo com o YOLO via Ultralytics, incluindo:
  - Preparação dos dados e anotações;
  - Treinamento do modelo;
  - Avaliação e interpretação dos resultados;